

Planificación del Motor de Matchmaking: Weighted Feature Distance con Escalabilidad a Machine Learning

Proyecto: MiLocal — Trust & Match

Stack: Go 1.23 + PostgreSQL 16 + React 19 + TypeScript 5.8

Repositorio: <https://github.com/felipe03/proyecto-inmobiliario>

31 de Julio de 2026

1. Identificación General del Motor

- **Nombre del motor:** Weighted Feature Distance (WFD).
- **Tipo de sistema:** Sistema de recomendación content-based con reglas duras.
- **Problema que resuelve:** Dado un emprendedor con un rubro comercial y preferencias personales (presupuesto, tamaño, ubicación), rankear inmuebles disponibles según compatibilidad técnica y necesidades específicas del negocio.
- **Objetivo general:** Proveer un puntaje de compatibilidad (*match score*) entre 0 y 1 que sea explicable, escalable por rubro, personalizable por preferencias de usuario, y migrable a modelos supervisados cuando se acumulen datos históricos de contratos firmados.
- **Hipótesis de diseño:** Cada rubro comercial tiene requerimientos técnicos distintos. Un sistema de pesos configurables por rubro, combinado con preferencias individuales del usuario, produce recomendaciones de mayor precisión que un ranking genérico por precio o ubicación.

2. Comprensión del Problema

- **Qué fenómeno modelan:** La compatibilidad entre las especificaciones técnicas de un inmueble comercial y los requerimientos ideales de un rubro de negocio.
- **Qué patrones buscan detectar:** Propiedades cuyas características técnicas (conexiones, dimensiones, flujo peatonal) maximicen el éxito operacional del rubro del emprendedor, penalizando aquellas cuyos usos permitidos excluyan el rubro.
- **Por qué es adecuado WFD:** Porque cada rubro define su propio espacio de características ideales, permitiendo que el sistema escale horizontalmente al agregar nuevos rubros sin modificar el motor de cálculo. La trazabilidad por dimensión garantiza explicabilidad total ante el usuario final.
- **Datos de entrada:** Especificaciones técnicas del inmueble (7 dimensiones) + preferencias del usuario (3 dimensiones) = vector de 10 features normalizadas en $[0, 1]$.

Preguntas de diseño:

- **¿Cómo escalar a nuevos rubros sin redeploy?** Actualmente los perfiles están hardcodeados en `profiles.go`. La Fase 2 migrará los perfiles a la tabla `rubro_profiles` en PostgreSQL, permitiendo CRUD administrativo.

- **¿Qué técnica de distancia funciona mejor?** Se evaluó Manhattan vs Euclidiana. Manhattan fue elegida porque las dimensiones son independientes entre sí (no hay correlación esperada entre gas y flujo peatonal) y es más interpretable para el breakdown.
- **¿Qué variables influyen más?** Depende del rubro. Para Gastronomía: trampa de grasas (25 %), flujo peatonal (25 %). Para Bodega: frente comercial (40 %), precio negociable (30 %). Esto valida el diseño de pesos por perfil.
- **¿Cómo afecta la normalización?** Todas las features se normalizan a $[0, 1]$ para que ninguna dimensión domine el score por diferencia de escala. Sin normalización, el precio (en millones) dominaría sobre features binarias.

3. Comprensión de los Datos

Basado en el esquema de base de datos (**properties**, **users**) y las entradas del endpoint POST /api/match:

- **Número de dimensiones:** 10 (7 técnicas + 3 de preferencias).
- **Número de rubros:** 6 configurados (Gastronomía, Belleza/Estética, Retail, Servicios, Bodega/Logística, Otro).
- **Datos faltantes:** Las preferencias de usuario son opcionales. Si no se proveen, el score de preferencias se neutraliza en 0.5 para no sesgar el resultado.
- **Tipo predominante:** Features numéricas continuas en $[0, 1]$, derivadas de datos mixtos (booleanos, ordinales, texto).

#	Dimensión	Tipo ori- gen	Encoding
0	Capacidad Eléctrica	Booleano	Básica = 0.0, Trifásica = 1.0
1	Conexión de Gas	Booleano	No = 0.0, Sí = 1.0
2	Conexión de Agua	Booleano	No = 0.0, Sí = 1.0
3	Trampa de Grasa	Booleano	No = 0.0, Sí = 1.0
4	Frente Comercial	Numérico (m)	$\min(\text{metros}/20, 1.0)$
5	Flujo Peatonal	Ordinal	Bajo = 0.33, Medio = 0.66, Alto = 1.0
6	Precio Negociable	Booleano	No = 0.0, Sí = 1.0
7	Ajuste Presupuesto	Función	$f(\text{ratio}(\text{precio}/\text{budget}))$, 4 niveles
8	Ajuste Tamaño	Función	$f(\text{mvsrango}[\text{min}, \text{max}])$, 3 niveles
9	Match Ubicación	Similitud texto	Fuzzy match de strings

4. Estado Actual del Motor (Julio 2026)

Componente	Estado
Extracción de features técnicas	Implementado. 7 dimensiones desde <code>model.Property</code> .
Extracción de features de usuario	Implementado. 3 dimensiones con neutralidad en 0.5 si no se proveen.
Perfiles de rubro	Implementado. 6 perfiles con vectores de ideales y pesos en <code>profiles.go</code> .
Motor de scoring	Implementado. Weighted Manhattan Distance + blending spec/user.
Hard rule 29 %	Implementado. Cap si rubro $\notin$ <code>permitted_uses</code> .
Explainability	Implementado. Breakdown de 10 dimensiones con ideal vs actual.
Persistencia de matches	Pendiente. Tabla <code>matches</code> creada pero no escrita.
Tests unitarios	Pendiente. Cero tests en el paquete <code>matcher/</code> .
Migración a ML	Pendiente. Arquitectura lista, falta recolección de labels.
Perfiles en base de datos	Pendiente. Hardcodeados, no administrables sin redeploy.

5. Plan de Escalamiento

- **Fase 1 — Perfiles en DB:** Migrar los 6 perfiles de `profiles.go` a una tabla `rubro_profiles` con columnas `rubro`, `dimension`, `ideal_value`, `weight`, `user_pref_weight`. Crear endpoint GET/PUT `/api/admin/profiles` para administración sin redeploy.
- **Fase 2 — Recolección de labels:** Almacenar cada match calculado en la tabla `matches` con `status` (`scored`, `contacted`, `closed_won`, `closed_lost`). Requiere ≥ 500 matches etiquetados para entrenamiento supervisado.
- **Fase 3 — Modelo supervisado:** Entrenar XGBoost con los mismos 10 features vectorizados + interacciones `rubro  $\times$  feature` + `precio/m2` vs mediana zonal. El modelo reemplaza `weightedDistance()` manteniendo interfaz `EngineResult`.
- **Fase 4 — Sistema híbrido:** Reglas como fallback, ML como scorer primario cuando confianza $> 0,7$. A/B testing entre motor de reglas y motor ML.

Fase	Complejidad	Objetivo
Perfiles en DB	Media	Permitir tunear pesos e ideales sin redeploy. Habilitar A/B testing de configuraciones.
Labels	Media	Registrar outcomes reales de matches para entrenar modelo supervisado.
XGBoost	Alta	Reemplazar reglas con modelo entrenado sobre datos reales de contratos.
Híbrido	Alta	Combinar reglas (explicabilidad) con ML (precisión) según confianza.

6. Plan de Perfiles (Configuración Actual y Futura)

6.1. Perfiles Actuales (hardcodeados en `profiles.go`)

Rubro	Potencia	Gas	Agua	Trampa	Frente	Tráfico
Gastronomía	0.80 (10 %)	1.00 (15 %)	1.00 (10 %)	1.00 (25 %)	0.25 (5 %)	0.80 (25 %)
Belleza	0.50 (10 %)	0.30 (5 %)	0.80 (25 %)	0.00 (5 %)	0.30 (15 %)	0.80 (30 %)
Retail	0.50 (5 %)	0.00 (0 %)	0.30 (5 %)	0.00 (0 %)	0.50 (30 %)	0.80 (30 %)
Servicios	0.60 (20 %)	0.30 (5 %)	0.60 (15 %)	0.20 (5 %)	0.20 (10 %)	0.50 (20 %)
Bodega	0.50 (10 %)	0.00 (0 %)	0.30 (5 %)	0.00 (0 %)	0.60 (40 %)	0.20 (15 %)
Otro	0.50 (15 %)	0.50 (15 %)	0.50 (15 %)	0.30 (10 %)	0.30 (10 %)	0.50 (20 %)

Cuadro 1: Vector ideal y peso (entre paréntesis) por dimensión técnica y rubro. Negociable completa las 7 dimensiones (no mostrado por espacio).

6.2. Migración a Base de Datos

```
CREATE TABLE rubro_profiles ( rubro VARCHAR(50), dimension VARCHAR(30), ideal_value
NUMERIC(3,2), weight NUMERIC(3,2), user_pref_weight NUMERIC(3,2), PRIMARY KEY (rubro,
dimension) );
```

- **Ventaja:** Administración vía API sin redeploy. A/B testing de configuraciones de pesos.
- **Desventaja:** Latencia adicional de 1 query por request de match. Mitigable con caché en memoria (carga al inicio, invalida al modificar).

7. Plan de Scoring

- **Algoritmo principal: Weighted Manhattan Distance**

- **Fórmula specScore:** $specScore = \frac{\sum w_i \cdot (1 - |actual_i - ideal_i|)}{\sum w_i}$

- **Fórmula userScore:** $userScore = \frac{\sum uw_j \cdot (1 - |userActual_j - 1,0|)}{\sum uw_j}$

- **Fórmula finalScore:** $finalScore = specScore \cdot (1 - \alpha) + userScore \cdot \alpha$

- **Objetivo:** Maximizar interpretabilidad. Cada dimensión contribuye de forma independiente y trazable al score final.

- **Regla dura: Cap de usos permitidos**

- **Condición:** Si `rubro ∉ property.permitted_uses` $finalScore = \min(finalScore, 0,29)$.
- **Objetivo:** Evitar recomendar inmuebles donde el rubro no está legalmente permitido. El 29 % es deliberadamente inferior al umbral psicológico de “30 % = malo” pero mantiene visibilidad.

8. Plan de Hiperparámetros

Hiperparámetro	Valor actual	Estrategia de ajuste
Pesos por dimensión (specs)	Definidos por rubro en <code>profiles.go</code>	Ajuste manual por expertos de dominio. Futuro: optimización con grid search sobre datos etiquetados.
Pesos de preferencias (user)	[0.4, 0.3, 0.3] para budget, size, location	Igual para todos los rubros. Podría ser específico por rubro en el futuro.
UserPrefWeight (α)	0.25–0.35 según rubro	Controla el balance specs vs preferencias. Valores más altos dan más peso a lo que el usuario quiere vs lo que el rubro necesita.
Cap de no permitidos	0.29	Constante. Podría ser configurable por rubro (ej. 0.15 para Gastronomía que es más restrictivo).
Neutralidad	0.5	Valor asignado cuando el usuario no provee preferencias. Centrado para no sesgar.

9. Plan Experimental

Exp.	Rubro	Escenario	Especificaciones del inmueble	Score esperado
E1	Gastronomía	Local ideal	Gas=Sí, Agua=Sí, Trampa=Sí, Trifásica, Alto tráfico, Frente=15m, Negociable=Sí	$> 0,85$
E2	Gastronomía	Sin gas ni trampa	Gas=No, Trampa=No, resto igual a E1	$0,40 - 0,55$
E3	Gastronomía	Bodega (no permitido)	Mismo inmueble de E1 pero <code>permitted_uses = [Bodega]</code>	$\leq 0,29$
E4	Bodega	Local con gran frente	Frente=20m, Negociable=Sí, Trifásica, resto mínimo	$> 0,80$
E5	Retail	Sin preferencias	specs variadas, sin <code>userPreferences</code>	$\text{userScore} \approx 0,5$
E6	Belleza	Budget exacto	$\text{price} \leq \text{maxBudget}$	$\text{Budget fit} = 1.0$
E7	Cualquiera	Precio $> 1,5 \times \text{budget}$	$\text{price} = 2\text{M}$, $\text{maxBudget} = 1\text{M}$	$\text{Budget fit} = 0.1$
E8	Cualquiera	Ubicación exacta	<code>preferredLocation = ubicación real del inmueble</code>	$\text{Location match} = 1.0$

Cuadro 2: Escenarios de validación manual para el motor de matchmaking.

10. Plan de Evaluación

Métrica	Qué evalúa	Resultado esperado
Precisión del score	Correspondencia entre puntaje numérico y percepción subjetiva del usuario/experto sobre la compatibilidad real.	Scores $> 0,80$ deben corresponder a matches que expertos consideren “altamente compatibles”. Scores $< 0,30$ deben ser matches inviables.
Cobertura de cap	El 29 % de cap debe activarse solo cuando el rubro no está en <code>permitted_uses</code> . Ni antes ni después.	100 % de precisión en la activación del cap. Sin falsos positivos ni falsos negativos.
Neutralidad	Cuando no hay preferencias de usuario, el <code>userScore</code> no debe sesgar el resultado final.	<code>userScore = 0.5</code> exactamente cuando los 3 campos de preferencias están ausentes.
Consistencia inter-rubro	Un mismo inmueble debe tener scores distintos para distintos rubros, reflejando la especialización.	La varianza de scores entre rubros para un mismo inmueble debe ser $> 0,15$.
Determinismo	Mismos inputs mismo score siempre.	100 % determinístico. Sin aleatoriedad.

Estas métricas fueron seleccionadas para evaluar la calidad del motor desde perspectivas de **precisión, robustez y consistencia**. Al ser un sistema determinístico basado en reglas, no se evalúa con métricas de clustering (silhouette, Davies-Bouldin) sino con métricas de sistema de recomendación: precisión del ranking, cobertura de reglas de negocio, y neutralidad ante datos faltantes.

11. Evaluación Comparativa de Enfoques

Se contrastó Weighted Feature Distance contra tres enfoques alternativos durante la fase de diseño:

Enfoque		Ventajas	Desventajas	Veredicto
Reglas if/else		Máxima explicabilidad. Sin dependencias matemáticas.	No escala: cada rubro nuevo requiere código nuevo. Imposible migrar a ML. Pesos implícitos e inajustables.	Descartado. Era el enfoque inicial del prototipo monolítico.
Cosine	Similarity	Simplicidad matemática. Bueno para texto.	No soporta pesos por dimensión. No diferencia specs de preferencias. La normalización implícita oscurece el breakdown.	Descartado. No cumple el requisito de explicabilidad por dimensión con pesos.
WFD	(elegido)	Pesos explícitos por rubro. Breakdown trazable. Arquitectura compatible con ML. Manhattan es interpretable.	Requiere definir perfiles manualmente. Los pesos iniciales dependen de criterio experto.	Seleccionado. Mejor balance explicabilidad/escalabilidad.
XGBoost / NN		Máxima precisión potencial. Aprende interacciones no lineales.	Requiere ≥ 500 datos etiquetados. Caja negra: pierde explicabilidad. Overfitting con pocos datos.	Reservado para Fase 4. No viable sin datos históricos.

12. Arquitectura del Código

Archivo	Responsabilidad	Dependencias
<code>matcher/profiles.go</code>	Define 6 rubros con vectores de ideales, pesos y α . Único archivo a modificar para ajustar perfiles.	Ninguna (solo tipos internos).
<code>matcher/engine.go</code>	Implementa <code>weightedDistance()</code> y <code>Score()</code> . Produce <code>EngineResult</code> con break-down.	<code>profiles.go</code> (lee perfiles).
<code>matcher/matcher.go</code>	API pública <code>Calculate()</code> . Extrae features de <code>model.Property</code> y <code>UserPreferences</code> . Orquesta el flujo.	<code>profiles.go</code> , <code>engine.go</code> .
<code>service/match.go</code>	Wrapper de servicio. <code>CalculateMatch()</code> y <code>GetMatchDetails()</code> .	<code>matcher/</code> , <code>repository/</code> .
<code>handler/assessment.go</code>	Handler HTTP. <code>CalculateMatchScore()</code> para POST <code>/api/match</code> .	<code>service/</code> , <code>model/</code> .

13. Plan de Visualización y Herramientas

Visualización	Objetivo
Barras de progreso por dimensión	Frontend muestra % de compatibilidad por cada dimensión técnica (ej. "Capacidad Eléctrica: 85 %").
Radar chart de specs	Comparar visualmente el perfil ideal del rubro vs el inmueble real en 7 ejes.
Badge de score global	Color-coded: verde ($> 0,80$), amarillo ($0,50 - 0,80$), rojo ($< 0,50$). Cap del 29 % siempre rojo.
Breakdown tabular	Tabla ordenada por contribución al score, mostrando ideal vs actual con indicador visual.
Comparación lado a lado	Dos inmuebles con sus radar charts superpuestos para decisión rápida.

Elemento	Herramienta
Lenguaje del motor	Go 1.23 (stdlib <code>math</code> , sin dependencias externas).
Base de datos	PostgreSQL 16 + <code>lib/pq</code> .
Vectorización de features	Funciones puras en Go. Sin dependencias de ML.
API	<code>net/http</code> stdlib con path params de Go 1.22+.
Frontend (visualización)	React 19 + TypeScript 5.8 + Recharts + motion.
Testing	<code>go test</code> (unit), <code>httptest</code> (integration), Vitest (frontend).
CI/CD	Docker Compose + Jenkins.
Documentación	Markdown + $\text{\LaTeX}$.

14. Integración con Módulos del Sistema

14.1. Módulos Actuales (Julio 2026)

Endpoint	Método	Relación con Matchmaking
<code>/api/match</code>	POST	Punto principal de entrada. Calcula score + breakdown.
<code>/api/properties</code>	GET	Fuente de inmuebles a rankear. Soporta filtro <code>?rubro=</code> .
<code>/api/properties/{id}</code>	GET	Detalle de inmueble individual para el cálculo.
<code>/api/assessment</code>	POST	Captura rubro y preferencias del usuario para alimentar el match.
<code>/api/users/{id}</code>	GET	Perfil del usuario con documentos y nivel de confianza.
<code>/api/auth/*</code>	POST/GET	Autenticación JWT. Requerida para persistir matches por usuario.

14.2. Módulos con IA — On Hold

Módulo	Estado
POST /api/chat	[PLACEHOLDER — Pendiente decisión de cliente sobre proveedor LLM] Chat conversacional con asistente estratégico. Actualmente implementado como proxy a Gemini 3 Flash Preview. Se requiere definir si se migra a DeepSeek V4 Pro, Claude, OpenAI, o se mantiene Gemini. El modelo final debe ser acordado con el cliente.
GET /api/trends	[PLACEHOLDER — Pendiente decisión de cliente sobre proveedor LLM y fuente de datos] Dashboard de tendencias de mercado. Depende de LLM + Google Search grounding para datos actualizados. Existen alternativas sin IA (ver docs/trends/ALTERNATIVAS_SIN_IA.md).
Visual Analyzer	[PLACEHOLDER — Pendiente definición de alcance] Análisis visual de fachadas. Requiere modelo multimodal con capacidades de visión. No implementado (solo placeholder en UI).

Nota sobre IA/LLM: Todas las funcionalidades que dependen de modelos de lenguaje o APIs de inteligencia artificial están en estado ON HOLD hasta que el cliente defina el proveedor y modelo a utilizar. Las alternativas bajo evaluación incluyen: Gemini (Google), DeepSeek V4 Pro, Claude (Anthropic), GPT-4o (OpenAI), y alternativas sin IA para el módulo de trends (detalladas en docs/trends/ALTERNATIVAS_SIN_IA.md).

15. Path a Machine Learning

15.1. Requisitos previos

- **Datos etiquetados:** ≥ 500 matches con outcome conocido (`closed_won` o `closed_lost`).
- **Features derivadas:** Interacciones rubro $\times$ feature, precio/m² vs mediana zonal, antigüedad del anuncio, tasa de contacto.
- **Infraestructura:** Pipeline de entrenamiento offline (no en el servidor de producción). Exportación de features desde PostgreSQL.

15.2. Estrategia de migración

1. **Shadow mode:** El modelo ML corre en paralelo al motor de reglas, registrando predicciones sin afectar al usuario. Se comparan scores durante 2–4 semanas.
2. **A/B test:** 10 % del tráfico recibe scores del modelo ML. Se mide tasa de conversión (contacto $\rightarrow$ visita $\rightarrow$ contrato).
3. **Cutover:** Si el modelo ML supera al motor de reglas en $> 5\%$ de tasa de conversión con significancia estadística ($p < 0,05$), se promueve a primary scorer. Las reglas quedan como fallback.

16. Cronograma del Proyecto

Semana	Fase	Entregables
Jul 1–7	Scaffolding	Esquema DB, estructura de paquetes Go, Docker Compose, definición de tipos.
Jul 8–14	Core CRUD + Motor WFD	Repositorios, handlers CRUD, 6 perfiles de rubro, engine.go, matcher.go.
Jul 15–21	API Match + Auth + Upload	Endpoint /api/match, autenticación JWT, file upload, middleware CORS.
Jul 22–29	Frontend + CI/CD	8 páginas SPA, integración real con API, Jenkins pipeline, mobile responsive.
Ago (plan) 1–7	Perfiles en DB + Testing	Migrar perfiles a tabla <code>rubro_profiles</code> , tests unitarios del matcher, tests de integración.
Ago (plan) 8–14	RBAC + Validación + Paginación	Control de acceso por rol, validación de inputs, paginación en /api/properties.
Ago (plan) 15–21	Persistencia + UI avanzada	Persistir matches, assessment y chat history. Radar charts, comparación lado a lado.
Ago (plan) 22–29	MVP Launch	Deploy productivo, monitoreo, documentación final, handoff.
Q4 (tent.) 2026	Recolección labels	Tabla matches con outcomes. ≥ 500 registros etiquetados.
Q1 (tent.) 2027	ML Phase 1	Entrenamiento XGBoost, shadow mode, A/B testing.

17. Referencias

- Ricci, F., Rokach, L., & Shapira, B. (2011). *Recommender Systems Handbook*. Springer. Capítulo 4: Content-Based Recommendation Systems.
- Chen, T., & Guestrin, C. (2016). XGBoost: A Scalable Tree Boosting System. *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*.
- McInnes, L., Healy, J., & Melville, J. (2018). UMAP: Uniform Manifold Approximation and Projection for Dimension Reduction. *Journal of Open Source Software*, 3(29), 861.
- Documentación interna del proyecto: docs/matchmaking/ALGORITMO_MATCHMAKING.md, docs/matchmaking/PLAN_MATCHMAKING.md.
- Go 1.23 Standard Library: net/http, math, crypto/rand. <https://pkg.go.dev/std>